

Library Management System (C++ - OOP)

This project demonstrates a Library Management System developed using Object-Oriented Programming (OOP) concepts in C++. The system efficiently manages books, members, borrowing records, issue and return operations, and search functionality.

1. Book Class

```
class Book {
public:
    int id;
    string title;
    string author;
    bool isIssued;

    Book(int i, string t, string a) {
        id = i;
        title = t;
        author = a;
        isIssued = false;
    }
};
```

2. Member Class

```
class Member {
public:
    int memberId;
    string name;

    Member(int id, string n) {
        memberId = id;
        name = n;
    }
};
```

3. Library Class

```
#include <iostream>
#include <vector>
using namespace std;

class Library {
private:
    vector<Book> books;
    vector<Member> members;

public:
    // Add Book
    void addBook(int id, string title, string author) {
        books.push_back(Book(id, title, author));
        cout << "Book added successfully!\n";
    }

    // Add Member
    void addMember(int id, string name) {
```

```

        members.push_back(Member(id, name));
        cout << "Member added successfully!\n";
    }

    // Issue Book
    void issueBook(int bookId) {
        for (auto &book : books) {
            if (book.id == bookId && !book.isIssued) {
                book.isIssued = true;
                cout << "Book issued successfully!\n";
                return;
            }
        }
        cout << "Book not available.\n";
    }

    // Return Book
    void returnBook(int bookId) {
        for (auto &book : books) {
            if (book.id == bookId && book.isIssued) {
                book.isIssued = false;
                cout << "Book returned successfully!\n";
                return;
            }
        }
        cout << "Invalid return.\n";
    }

    // Search by Title
    void searchByTitle(string title) {
        for (auto &book : books) {
            if (book.title == title) {
                cout << "Found: " << book.title << " by " << book.author << endl;
                return;
            }
        }
        cout << "Book not found.\n";
    }

    // Search by Author
    void searchByAuthor(string author) {
        for (auto &book : books) {
            if (book.author == author) {
                cout << "Found: " << book.title << endl;
            }
        }
    }

    // Display Books
    void displayBooks() {
        for (auto &book : books) {
            cout << book.id << " | " << book.title << " | "
                << book.author << " | "
                << (book.isIssued ? "Issued" : "Available") << endl;
        }
    }
};

```

4. Main Function (User Menu)

```

int main() {
    Library lib;
    int choice;

    do {
        cout << "\n--- Library Menu ---\n";
        cout << "1. Add Book\n2. Add Member\n3. Issue Book\n";
    }
}

```

```

        cout << "4. Return Book\n5. Search by Title\n";
        cout << "6. Search by Author\n7. Display Books\n0. Exit\n";
        cout << "Enter choice: ";
        cin >> choice;

        int id;
        string title, author, name;

        switch (choice) {
        case 1:
            cout << "Enter ID, Title, Author: ";
            cin >> id >> title >> author;
            lib.addBook(id, title, author);
            break;

        case 2:
            cout << "Enter Member ID and Name: ";
            cin >> id >> name;
            lib.addMember(id, name);
            break;

        case 3:
            cout << "Enter Book ID: ";
            cin >> id;
            lib.issueBook(id);
            break;

        case 4:
            cout << "Enter Book ID: ";
            cin >> id;
            lib.returnBook(id);
            break;

        case 5:
            cout << "Enter Title: ";
            cin >> title;
            lib.searchByTitle(title);
            break;

        case 6:
            cout << "Enter Author: ";
            cin >> author;
            lib.searchByAuthor(author);
            break;

        case 7:
            lib.displayBooks();
            break;
        }

        } while (choice != 0);

    return 0;
}

```

Features Implemented

- ✓ Book & Member classes
- ✓ Add books and members
- ✓ Issue and return books
- ✓ Search by title and author
- ✓ Display all books
- ✓ Menu-driven interface

Possible Enhancements

- Store data in files (persistent storage)
- Add due dates and fine calculation
- Use ID validation for members issuing books
- Improve search with partial matching
- GUI using Qt or web interface